

Проект: Линейные модели в машинном обучении

Цель проекта

- Разработать модель машинного обучения, которая поможет управлять рисками и принимать объективное решение о покупке.
 - Первая будет прогнозировать возможный удой коровы (целевой признак Удой);
 - Вторая — рассчитывать вероятность получить вкусное молоко от коровы (целевой признак Вкус молока).

План проекта

1. Загрузка и изучение данных

- Загрузить данные из файлов:
 - /datasets/ferma_main.csv
 - /datasets/ferma_dad.csv
 - /datasets/cow_buy.csv
- Вывести первые строки и информацию, дополнительно проверка стандартизации оформления данных и наименование столбцов.

1. Предобработка данных

- Проверка данных на наличие пропусков и дубликатов. Проверка корректности типов данных у каждого признака.

1. Исследовательский анализ данных

- Проведение статистического анализа всех признаков.
- Построение графиков для каждого признака.
- Вывод о данных:
 - Наличие выбросов, необычное распределение количественных признаков и другие особенности.
 - Идеи по улучшению данных.

1. Проведение корреляционного анализа признаков в датасете

- Изучение взаимосвязи между признаками: расчет коэффициентов корреляции между всеми признаками.
- Построение диаграммы рассеяния scatterplot для признака "Удой", кг и всех количественных признаков с учетом значения категориальных признаков.
- Выводы о взаимосвязи между признаками:
 - Какой тип взаимосвязи между целевым и входными признаками?
 - Есть ли нелинейные связи между целевым и входными признаками? Нужно ли вводить в модель дополнительные признаки, чтобы это учесть?
 - Наблюдается ли мультиколлинеарность между входными признаками?

1. Задача регрессии

- Подготовить данные датасета ferma_main.csv и обучить на них первую модель LinearRegression.
 - Использовать в качестве целевого признак Удой, кг.
 - Отобрать признаки, которые подходят для модели линейной регрессии, аргументировать выбор. Опирайтесь не только на наличие или отсутствие линейной

- связи между признаками, но и на причинно-следственные связи между удоем коров и группами признаков из описания датасета.
- Разбить данные на тренировочную и тестовую выборки.
 - Кодировать категориальные признаки с помощью OneHotEncoder.
 - Масштабировать количественные признаки с помощью StandardScaler.
 - Обучить модель линейной регрессии на подготовленных данных.
 - Оценить качество модели на тестовой выборке: рассчитайте R^2 , проанализировать остатки с помощью графиков.
 - Оформить выводы, указать:
 - значение R^2 модели;
 - какие выводы следуют из анализа остатков;
 - нужны ли дополнительные исследования.
- Подготовить данные с учётом связи входных признаков с целевым и обучить на них вторую модель LinearRegression.
 - Оценить диаграммы рассеяния: возможно их можно увидеть, что некоторые входные признаки связаны с целевым нелинейно. Исключить нелинейность, чтобы эти признаки были информативны для линейной регрессии. Это можно сделать:
 - При обнаружении, что СПО и Удой связаны нелинейно. На их диаграмме рассеяния наблюдения сгруппированы в два кластера, причём есть такое значение СПО, которое можно считать границей между кластерами. Чтобы устранить нелинейность, преобразовать СПО в категориальный бинарный признак, используя эту границу.
 - При обнаружении, что ЭКЕ и Удой также связаны нелинейно. Нелинейность между ними можно исключить, если ввести новый признак ЭКЕ в квадрате.
 - Повторить шаги из пункта 1 и создать вторую модель: подготовить данные, обучить модель, оценить её качество и написать выводы.
 - Добавить в данные новый признак и обучить на них третью модель LinearRegression.
 - Добавить признак "Имя Папы" из файла ferma_dad.csv, соединив его с остальными данными ferma_main.csv по ключу.
 - Устранить нелинейность между признаками.
 - Повторить шаги из пункта 1 и создать третью модель: подготовить данные, обучить модель, оценить её качество и написать выводы.
 - Сравнить качество трёх моделей линейной регрессии, сопоставить их R^2 и остатки.
 - Оценить качество каждой модели по трём метрикам: MSE, MAE, RMSE. Использовать функции для расчёта метрик.
 - Выбрать лучшую модель;
 - Указать доверительный интервал прогноза лучшей модели. Для этого рассчитать квантили ошибки модели на тренировочных данных.
 - С помощью лучшей модели спрогнозировать удои коров, которых фермер хочет купить. Данные о них находятся в датасете cow_buy.csv.
 - Добавить недостающие признаки ЭКЕ (Энергетическая кормовая единица), Сырой протеин, г и СПО (Сахаро-протеиновое соотношение): взять из датасета ferma_main.csv средние значения каждого из признаков и увеличить их на 5%.
 - Выполнить подготовку данных, аналогичную той, которую делали для тренировочной выборки.
 - Получить прогноз удоя коров.

1. Задача классификации

- Подготовить данные датасета ferma_main.csv и обучить на них модель LogisticRegression.
 - Использовать в качестве целевого признака Вкус молока.

- При необходимости преобразовать имеющиеся или добавить новые признаки.
- Разбить данные на тренировочную и тестовую выборки.
- Кодировать категориальные признаки с помощью OneHotEncoder.
- Масштабировать количественные признаки с помощью StandardScaler.
- Обучить модель логистической регрессии на подготовленных данных.
- Оценить качество модели на тестовой выборке: рассчитать метрики accuracy, recall, precision.
- Построить матрицу ошибок.
- Проанализировать модель логистической регрессии.
 - Объяснить, какая ошибка критичнее всего при покупке коров: первого или второго рода. Помнить, что главное для заказчика — вкусное молоко. Сформулировать ответ с учётом метрик recall и precision, выбрать более важную из них.
 - Свести критичную ошибку к нулю: для этого изменить порог принадлежности к классам. Ещё раз рассчитать значения метрик recall и precision. Какое значение порога мы выбрали? Как изменились значения метрик?
 - Оформить выводы: определить, что нужно сделать дальше, исходя из анализа модели?
- С помощью обученной модели спрогнозировать вкус молока коров, которых фермер хочет купить. Данные о них находятся в датасете cow_buy.csv.
 - Выполнить подготовку данных, аналогичную той, которую делали для тренировочной выборки.
 - Получить прогноз вкуса молока коров.

1. Сделать итоговые выводы

- Добавить прогнозы вкуса молока и удоя коров «ЭкоФермы» в датафрейм с коровами, покупку которых рассматривает фермер.
- Отбрать коров, для которых спрогнозирован удой более 6000 килограммов в год и вкусное молоко.
- Коротко описать результаты работы моделей для фермера.
 - Сколько коров он может купить из стада на продажу с минимальным риском для себя? Почему?
 - Какие ещё рекомендации можно ему дать?
- Сделать выводы о моделях линейной регрессии и логистической регрессии.
 - Какие способы улучшения качества регрессионной модели вы использовали в этой задаче?
 - Возможно ли ещё улучшить результаты регрессионной модели? Если можно, то как? Если нет, то почему?
 - Какие метрики моделей классификации, в том числе логистической регрессии, важны при анализе рисков или экономических расчётов?

Описание данных

1. Файл ferma_main.csv содержит данные о стаде фермера на текущий момент. Описание данных:

- id — уникальный идентификатор коровы.
- Удой, кг — масса молока, которую корова даёт в год (в килограммах).
- ЭКЕ (Энергетическая кормовая единица) — показатель питательности корма коровы.
- Сырой протеин, г — содержание сырого протеина в корме (в граммах).
- СПО (Сахаро-протеиновое соотношение) — отношение сахара к протеину в корме коровы.

- Порода — порода коровы.
 - Тип пастбища — ландшафт лугов, на которых паслась корова.
 - Порода папы_быка — порода папы коровы.
 - Жирность, % — содержание жиров в молоке (в процентах).
 - Белок, % — содержание белков в молоке (в процентах).
 - Вкус молока — оценка вкуса по личным критериям фермера, бинарный признак (вкусно, не вкусно).
 - Возраст — возраст коровы, бинарный признак (менее_2_лет, более_2_лет).
 - Признаки в датасете `ferma_main.csv` можно разделить на группы.
 - Характеристики коровы: `id`, Порода, порода папы_быка, Возраст.
 - Характеристики корма: ЭКЕ (Энергетическая кормовая единица), Сырой протеин, г, СПО (Сахаро-протеиновое соотношение).
 - Характеристика пастбища: Тип пастбища.
 - Характеристики молока: Удой, кг, Жирность,%, Белок,%, Вкус молока.
 - Такое разделение поможет вам понять, как взаимосвязаны признаки в датасете.
1. Файл `ferma_dad.csv` хранит имя папы каждой коровы в стаде фермера. Описание данных:
- `id` — уникальный идентификатор коровы.
 - Имя Папы — имя папы коровы.
1. Файл `cow_buy.csv` — это данные о коровах «ЭкоФермы», которых фермер хочет изучить перед покупкой. Описание данных:
- Порода — порода коровы.
 - Тип пастбища — ландшафт лугов, на которых паслась корова.
 - Порода папы_быка — порода папы коровы.
 - Имя_папы — имя папы коровы.
 - Текущая_жирность,% — содержание жиров в молоке (в процентах).
 - Текущий_уровень_белок,% — содержание белков в молоке (в процентах).
 - Возраст — возраст коровы, бинарный признак (менее_2_лет, более_2_лет).
 - Данные в файлах `ferma_main.csv` и `cow_buy.csv` похожи, но у датасета `cow_buy.csv` несколько особенностей.
 - Содержание белков и жиров в молоке указано на момент продажи — когда «ЭкоФерма» давала коровам свой корм.
 - Параметры корма ЭКЕ (Энергетическая кормовая единица), Сырой протеин, г и СПО (Сахаро-протеиновое соотношение) отсутствуют. Технологи заказчика пересмотрели подход к кормлению: для новых коров планируется увеличить значения каждого из этих параметров на 5%.
 - Кроме того, отсутствуют признаки Удой, кг и Вкус молока. Это целевые признаки, именно их предстоит предсказать.

1. Загрузка данных

Загружаем необходимые библиотеки.

In [1]:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import math
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler
from scipy.sparse import hstack
```

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, recall_score, precision_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import precision_recall_curve
```

In [2]: `pip install phik`

```
Requirement already satisfied: phik in /opt/conda/lib/python3.9/site-packages (0.12.5)
Requirement already satisfied: numpy>=1.18.0 in /opt/conda/lib/python3.9/site-packages
(from phik) (1.21.1)
Requirement already satisfied: pandas>=0.25.1 in /opt/conda/lib/python3.9/site-packages
(from phik) (1.2.4)
Requirement already satisfied: joblib>=0.14.1 in /opt/conda/lib/python3.9/site-packages
(from phik) (1.1.0)
Requirement already satisfied: scipy>=1.5.2 in /opt/conda/lib/python3.9/site-packages
(from phik) (1.9.1)
Requirement already satisfied: matplotlib>=2.2.3 in /opt/conda/lib/python3.9/site-packa
ges (from phik) (3.3.4)
Requirement already satisfied: pyparsing!=2.0.4,!=2.1.2,!=2.1.6,>=2.0.3 in /opt/conda/l
ib/python3.9/site-packages (from matplotlib>=2.2.3->phik) (2.4.7)
Requirement already satisfied: kiwisolver>=1.0.1 in /opt/conda/lib/python3.9/site-packa
ges (from matplotlib>=2.2.3->phik) (1.4.4)
Requirement already satisfied: python-dateutil>=2.1 in /opt/conda/lib/python3.9/site-pa
ckages (from matplotlib>=2.2.3->phik) (2.8.1)
Requirement already satisfied: pillow>=6.2.0 in /opt/conda/lib/python3.9/site-packages
(from matplotlib>=2.2.3->phik) (8.4.0)
Requirement already satisfied: cycler>=0.10 in /opt/conda/lib/python3.9/site-packages
(from matplotlib>=2.2.3->phik) (0.11.0)
Requirement already satisfied: pytz>=2017.3 in /opt/conda/lib/python3.9/site-packages
(from pandas>=0.25.1->phik) (2021.1)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.9/site-packages (from
python-dateutil>=2.1->matplotlib>=2.2.3->phik) (1.16.0)
Note: you may need to restart the kernel to use updated packages.
```

In [3]: `import phik`

Далее загружаем и изучаем датафреймы.

In [4]: `ferma_main = pd.read_csv('/datasets/ferma_main.csv', sep = ';')
ferma_dad = pd.read_csv('/datasets/ferma_dad.csv', sep = ';')
cow_buy = pd.read_csv('/datasets/cow_buy.csv', sep = ';')`

In [5]: `display(ferma_main.head(3))
display(ferma_dad.head(3))
display(cow_buy.head(3))`

	id	Удой, кг	ЭКЕ (Энергетическая кормовая единица)	Сырой протеин, г	СПО (Сахаро-протеиновое соотношение)	Порода	Тип пастбища	порода папы_быка	Жи
0	1	5863	14,2	1743	0,89	Вис Бик Айдиал	Равнинное	Айдиал	
1	2	5529	12,8	2138	0,89	Вис Бик Айдиал	Равнинные	Соверин	
2	3	5810	14	1854	0,885	РефлешнСоверинг	Холмистое	Соверин	

	id	Имя Папы
0	1	Буйный
1	2	Соловчик
2	3	Барин

	Порода	Тип пастбища	порода папы_быка	Имя_папы	Текущая_жирность, %	Текущий_уровень_белка
0	Вис Бик Айдиал	холмистое	Айдиал	Геркулес	3,58	3,58
1	Вис Бик Айдиал	равнинное	Соверин	Буйный	3,54	3,54
2	РефлешнСоверинг	равнинное	Соверин	Барин	3,59	3,59

Переименуем столбцы во всех датафреймах, для удобства работы с ними.

```
In [6]: ferma_main = ferma_main.rename(columns={
    'Удой, кг': 'Milk',
    'ЭКЕ (Энергетическая кормовая единица)': 'Energy',
    'Сырой протеин, г': 'Protein',
    'СПО (Сахаро-протеиновое соотношение)': 'SP0',
    'Порода': 'Breed',
    'Тип пастбища': 'Pasture',
    'порода папы_быка': 'Father breed',
    'Жирность, %': 'Fat',
    'Белок, %': 'Prot in milk',
    'Вкус молока': 'Flavor',
    'Возраст': 'Age'
})

ferma_dad = ferma_dad.rename(columns={'Имя Папы': 'Father name'})

cow_buy = cow_buy.rename(columns={
    'Порода': 'Breed',
    'Тип пастбища': 'Pasture',
    'порода папы_быка': 'Father breed',
    'Имя_папы': 'Father name',
    'Текущая_жирность, %': 'Fat',
    'Текущий_уровень_белок, %': 'Prot in milk',
    'Возраст': 'Age'
})

cow_buy = cow_buy.replace({'холмистое': 'Холмистое', 'равнинное': 'Равнинное'})
```

```
In [7]: # проверка
print('Столбцы ferma_main:', ferma_main.columns)
print('--' * 50)
print('Столбцы ferma_dad:', ferma_dad.columns)
print('--' * 50)
print('Столбцы cow_buy:', cow_buy.columns)
```

```
Столбцы ferma_main: Index(['id', 'Milk', 'Energy', 'Protein', 'SP0', 'Breed', 'Pastur
e',
    'Father breed', 'Fat', 'Prot in milk', 'Flavor', 'Age'],
    dtype='object')
```

```
-----
Столбцы ferma_dad: Index(['id', 'Father name'], dtype='object')
```

```
-----
Столбцы cow_buy: Index(['Breed', 'Pasture', 'Father breed', 'Father name', 'Fat',
    'Prot in milk', 'Age'],
    dtype='object')
```

Вывод:

Данные не плохие. Столбцы преименовали для удобства. Так же изменили регистр первой буквы в столбце тип пастбища(Pasture).

2. Предобработка данных

Проверим данные на наличие пропусков и дубликатов. А так же проверим тип данных.

```
In [8]: print('Пропуски в ferma_main:', ferma_main.isna().sum())
print('-' * 50)
print('Пропуски в ferma_dad:', ferma_dad.isna().sum())
print('-' * 50)
print('Пропуски в cow_buy:', cow_buy.isna().sum())
```

```
Пропуски в ferma_main: id          0
Milk          0
Energy        0
Protein        0
SP0           0
Breed         0
Pasture       0
Father breed   0
Fat           0
Prot in milk   0
Flavor        0
Age           0
dtype: int64
```

```
-----
Пропуски в ferma_dad: id          0
Father name    0
dtype: int64
```

```
-----
Пропуски в cow_buy: Breed         0
Pasture        0
Father breed    0
Father name     0
Fat            0
Prot in milk    0
Age            0
dtype: int64
```

Пропуски не обнаружены.

```
In [9]: print('Дубликаты ferma_main:', ferma_main.duplicated().sum())
print('-' * 50)
print('Дубликаты ferma_dad:', ferma_dad.duplicated().sum())
print('-' * 50)
print('Дубликаты cow_buy:', cow_buy.duplicated().sum())
```

```
Дубликаты ferma_main: 5
```

```
-----
Дубликаты ferma_dad: 0
```

```
-----
Дубликаты cow_buy: 4
```

```
In [10]: ferma_main = ferma_main.drop_duplicates().reset_index(drop = True)
cow_buy = cow_buy.drop_duplicates().reset_index(drop = True)
```

```
In [11]: # проверка
print('Дубликаты ferma_main:', ferma_main.duplicated().sum())
print('Дубликаты cow_buy:', cow_buy.duplicated().sum())
```

```
Дубликаты ferma_main: 0
Дубликаты cow_buy: 0
```

Дубликаты удалены. Сделаем проверку неявных дубликатов.

```
In [12]: print(ferma_main.nunique())
```

```
id          629
Milk        528
Energy       55
Protein     401
SP0         12
Breed        2
```

```

Pasture      3
Father breed  3
Fat          50
Prot in milk 15
Flavor       2
Age          2
dtype: int64

```

Видим что, значений в столбце Pasture - 3, а должно быть 2 и Father breed аналогично.

```

In [13]: print(ferma_main['Pasture'].unique())
         print(ferma_main['Father breed'].unique())

```

```

['Равнинное' 'Равнинные' 'Холмистое']
['Айдиал' 'Соверин' 'Айдиалл']

```

```

In [14]: ferma_main['Pasture'] = ferma_main['Pasture'].replace({'Равнинные': 'Равнинное'})
         ferma_main['Father breed'] = ferma_main['Father breed'].replace({'Айдиалл': 'Айдиал'})

```

```

In [15]: print(ferma_main.nunique())

```

```

id          629
Milk        528
Energy      55
Protein     401
SP0         12
Breed       2
Pasture     2
Father breed 2
Fat         50
Prot in milk 15
Flavor      2
Age         2
dtype: int64

```

Переходим к просмотру типов данных.

```

In [16]: display('Типы данных ferma_main:', ferma_main.dtypes)
         print('-' * 50)
         display('Типы данных ferma_dad:', ferma_dad.dtypes)
         print('-' * 50)
         display('Типы данных cow_buy:', cow_buy.dtypes)

```

```

'Типы данных ferma_main:'
id          int64
Milk        int64
Energy      object
Protein     int64
SP0         object
Breed       object
Pasture     object
Father breed object
Fat         object
Prot in milk object
Flavor      object
Age         object
dtype: object

```

```

-----
'Типы данных ferma_dad:'
id          int64
Father name object
dtype: object

```

```

-----
'Типы данных cow_buy:'
Breed       object
Pasture     object
Father breed object
Father name object
Fat         object

```

```
Prot in milk    object
Age            object
dtype: object
```

```
In [17]: ferma_main['Energy'] = ferma_main['Energy'].str.replace(',', '.').astype(float)
ferma_main['SP0'] = ferma_main['SP0'].str.replace(',', '.').astype(float)
ferma_main['Fat'] = ferma_main['Fat'].str.replace(',', '.').astype(float)
ferma_main['Prot in milk'] = ferma_main['Prot in milk'].str.replace(',', '.').astype(float)
ferma_main['Flavor'] = ferma_main['Flavor'].map({'вкусно': 1, 'не вкусно': 0})
ferma_main['Age'] = ferma_main['Age'].map({'более_2_лет': 1, 'менее_2_лет': 0})

cow_buy['Fat'] = cow_buy['Fat'].str.replace(',', '.').astype(float)
cow_buy['Prot in milk'] = cow_buy['Prot in milk'].str.replace(',', '.').astype(float)
cow_buy['Age'] = cow_buy['Age'].map({'более_2_лет': 1, 'менее_2_лет': 0})
```

```
In [18]: # проверка
display('Типы данных ferma_main:', ferma_main.dtypes)
print('-' * 50)
display('Типы данных ferma_dad:', ferma_dad.dtypes)
print('-' * 50)
display('Типы данных cow_buy:', cow_buy.dtypes)
```

```
'Типы данных ferma_main:'
id          int64
Milk        int64
Energy      float64
Protein     int64
SP0         float64
Breed       object
Pasture     object
Father breed object
Fat         float64
Prot in milk float64
Flavor      int64
Age         int64
dtype: object
```

```
'Типы данных ferma_dad:'
id          int64
Father name object
dtype: object
```

```
'Типы данных cow_buy:'
Breed       object
Pasture     object
Father breed object
Father name object
Fat         float64
Prot in milk float64
Age         int64
dtype: object
```

Вывод по шагу 2:

- Пропуски обнаружены не были.
- Дубликаты явные и неявные устранены.
- Типы данных отредактированы.

3. Исследовательский анализ данных

Делаем статистический анализ.

```
In [19]: display(ferma_main.describe())
display(ferma_main.describe(exclude=['number']))
```

	id	Milk	Energy	Protein	SPO	Fat	Prot in milk	FI
count	629.00000	629.000000	629.000000	629.000000	629.000000	629.000000	629.000000	629.000
mean	315.00000	6188.750397	14.543879	1923.364070	0.913116	3.603657	3.075671	0.585
std	181.72094	1644.795489	1.306408	182.956251	0.032203	0.168656	0.002549	0.49%
min	1.00000	5204.000000	10.900000	1660.000000	0.840000	2.990000	3.069000	0.000
25%	158.00000	5751.000000	13.500000	1771.000000	0.890000	3.590000	3.074000	0.000
50%	315.00000	6133.000000	14.700000	1888.000000	0.930000	3.650000	3.076000	1.000
75%	472.00000	6501.000000	15.600000	2062.000000	0.940000	3.720000	3.077000	1.000
max	629.00000	45616.000000	16.800000	2349.000000	0.960000	3.750000	3.085000	1.000

	Breed	Pasture	Father breed
count	629	629	629
unique	2	2	2
top	Вис Бик Айдиал	Холмистое	Айдиал
freq	373	351	318

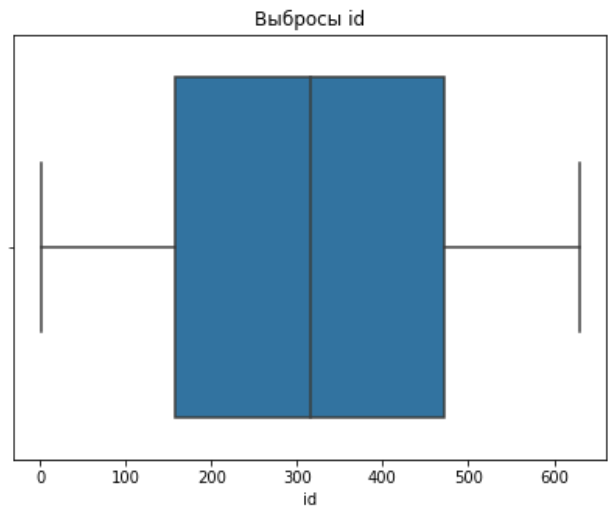
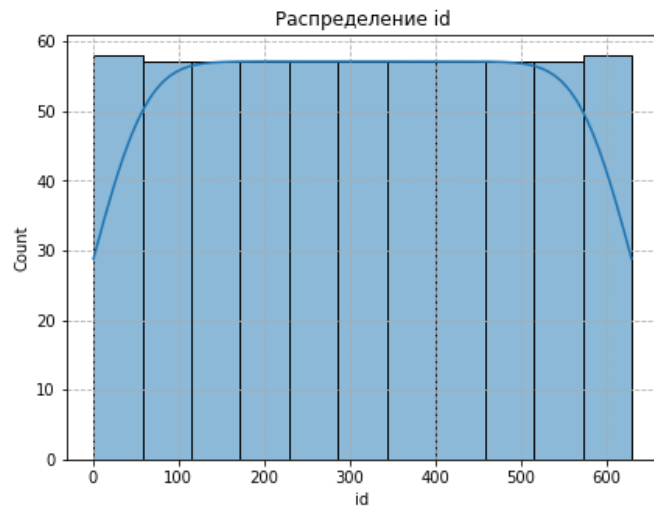
```
In [20]: # заменили аномальное значение, на среднее
ferma_main['Milk'] = ferma_main['Milk'].replace(45616, 6188)
```

```
In [21]: num_cols = ferma_main.select_dtypes(include = 'number').columns.tolist()
cat_cols = ferma_main.select_dtypes(include = 'object').columns.tolist()

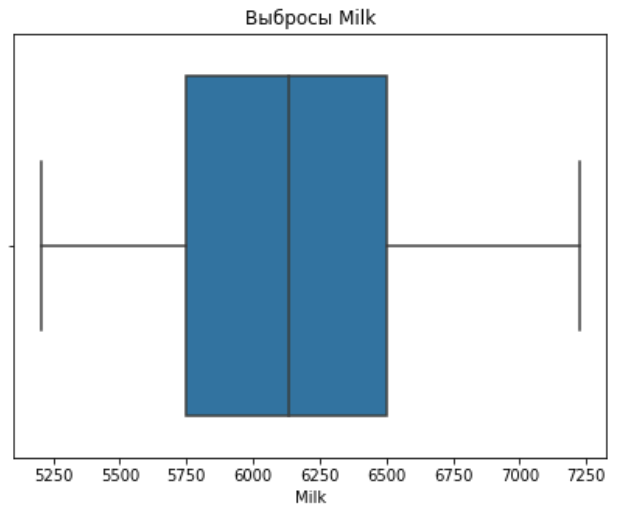
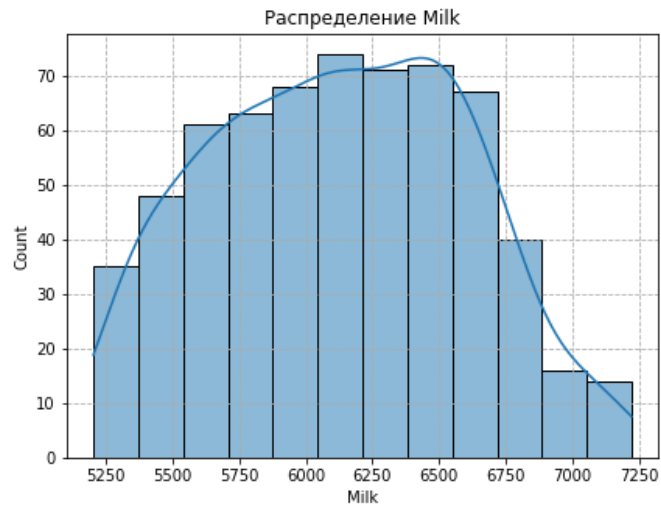
for col in num_cols:
    plt.figure(figsize=(15, 5))
    plt.subplot(1, 2, 1)
    sns.histplot(ferma_main[col], kde=True)
    plt.title(f'Распределение {col}')
    plt.grid(True, linestyle = '--')
    plt.subplot(1, 2, 2)
    sns.boxplot(x=ferma_main[col])
    plt.title(f'Выбросы {col}')
    plt.show()

    Q1 = ferma_main[col].quantile(0.25)
    Q3 = ferma_main[col].quantile(0.75)
    IQR = Q3 - Q1
    outliers = ferma_main[(ferma_main[col] < (Q1 - 1.75*IQR)) | (ferma_main[col] > (Q3
    print(f'{col}: {len(outliers)} выбросов')

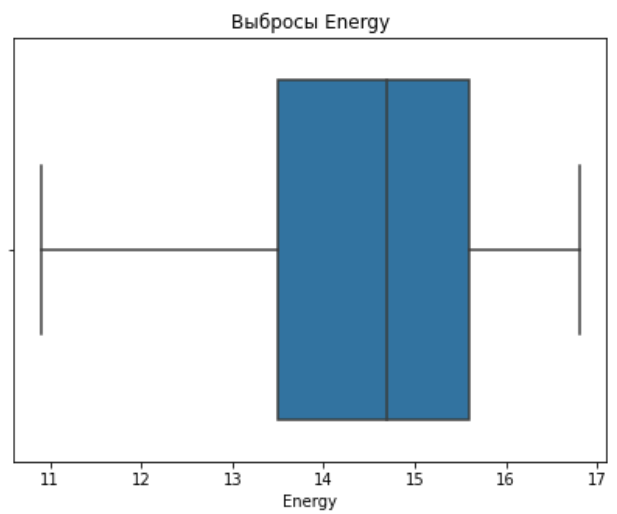
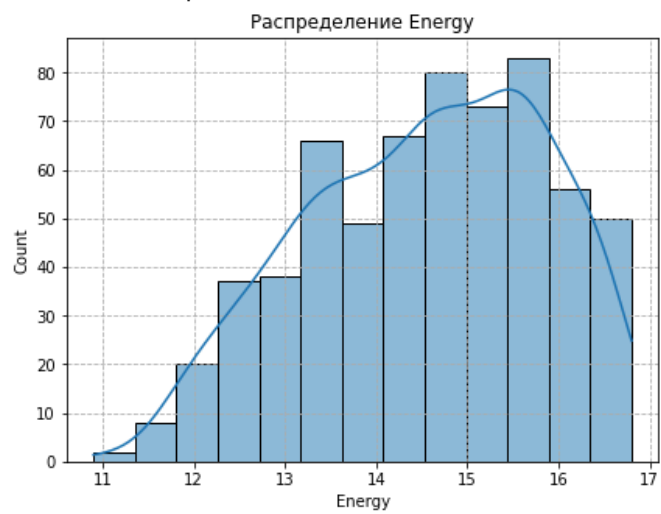
for col in cat_cols:
    plt.figure(figsize=(10, 4))
    sns.countplot(x=col, data=ferma_main)
    plt.title(f'Распределение {col}')
    plt.grid(True, linestyle = '--')
    plt.xticks(rotation=45)
    plt.show()
    print(ferma_main[col].value_counts());
```



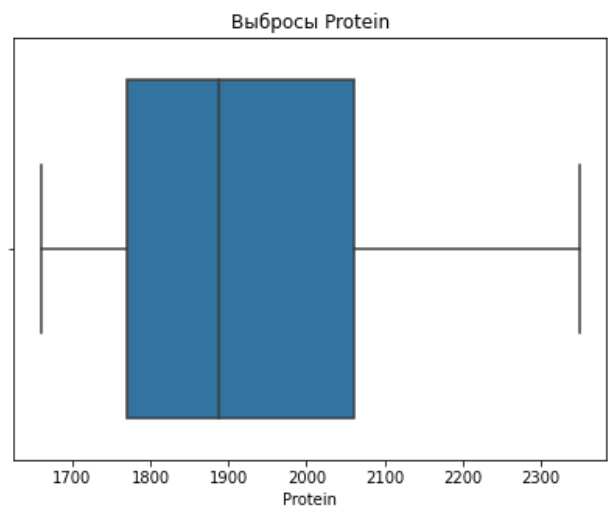
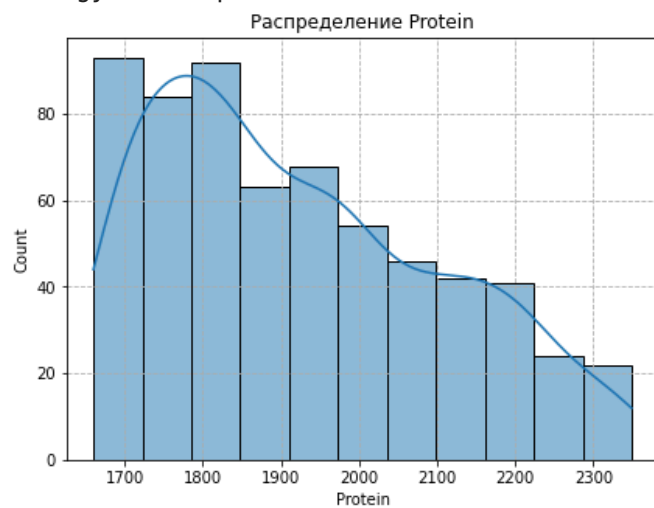
id: 0 выбросов



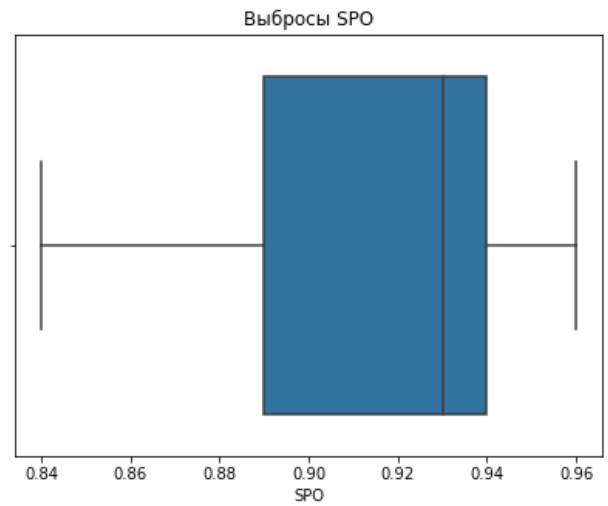
Milk: 0 выбросов



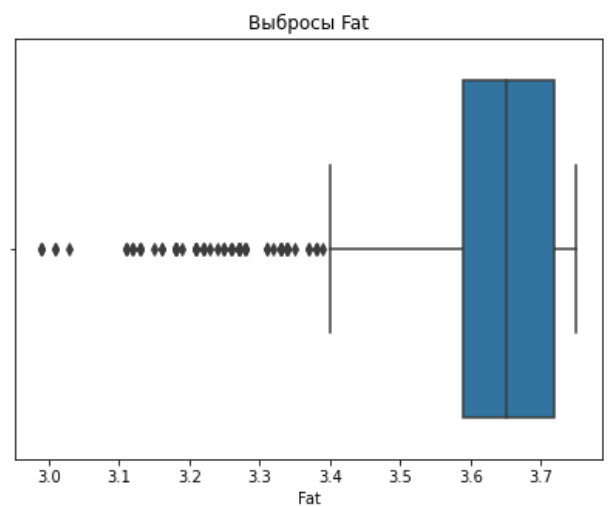
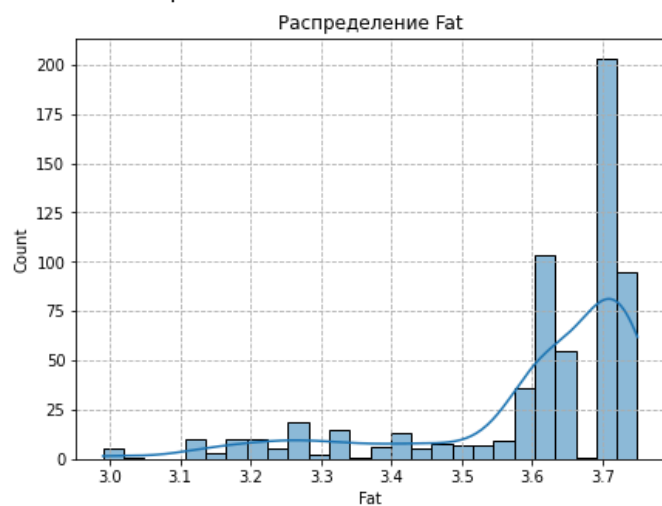
Energy: 0 выбросов



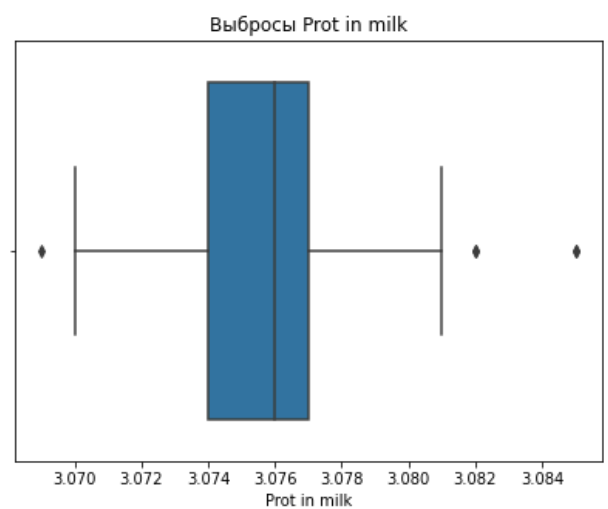
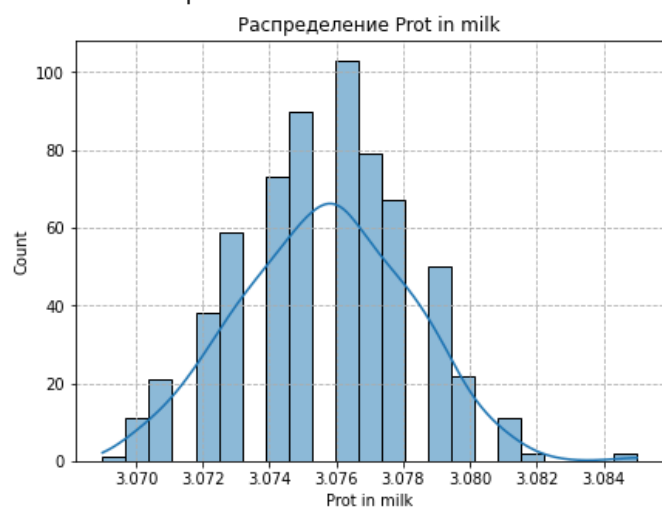
Protein: 0 выбросов



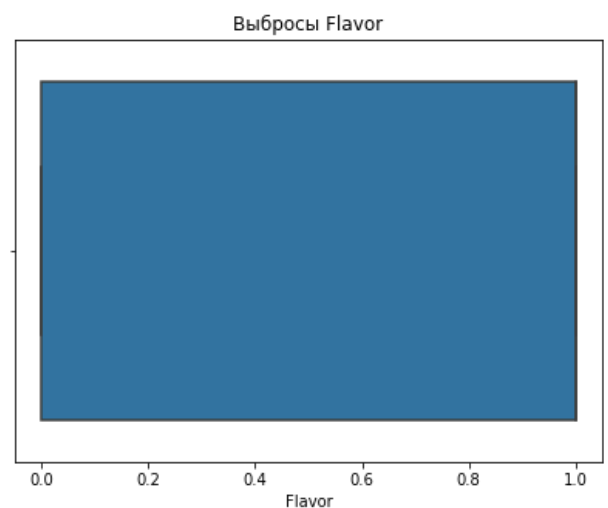
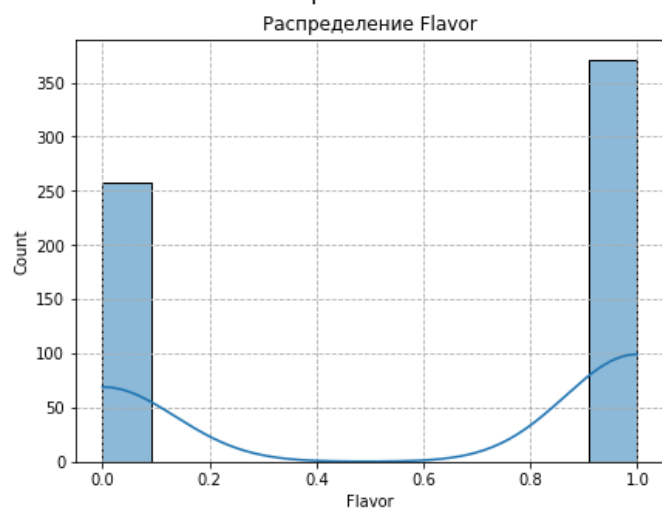
SPO: 0 выбросов



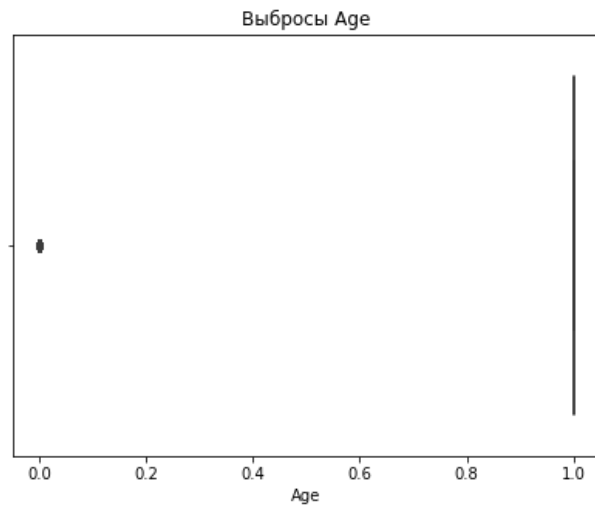
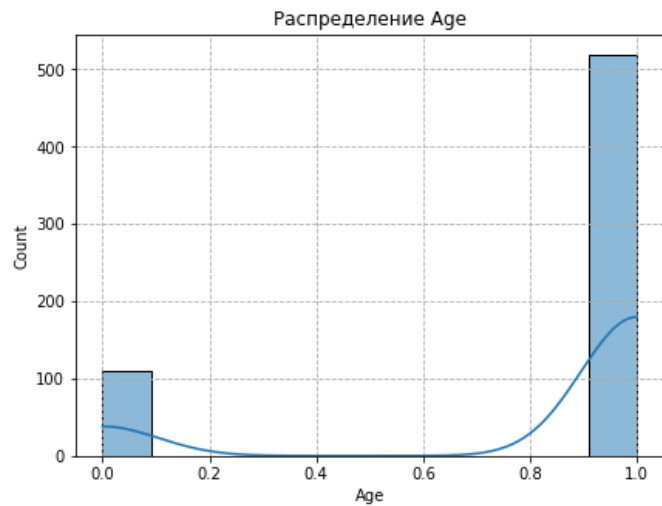
Fat: 81 выбросов



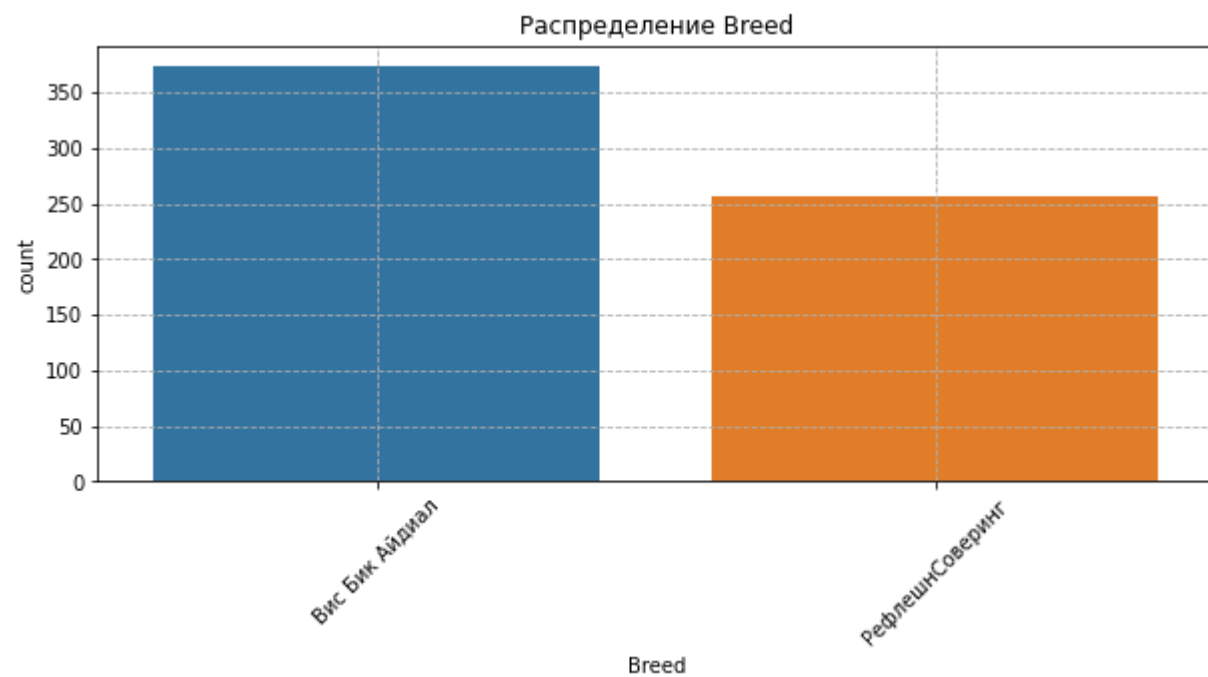
Prot in milk: 2 выбросов



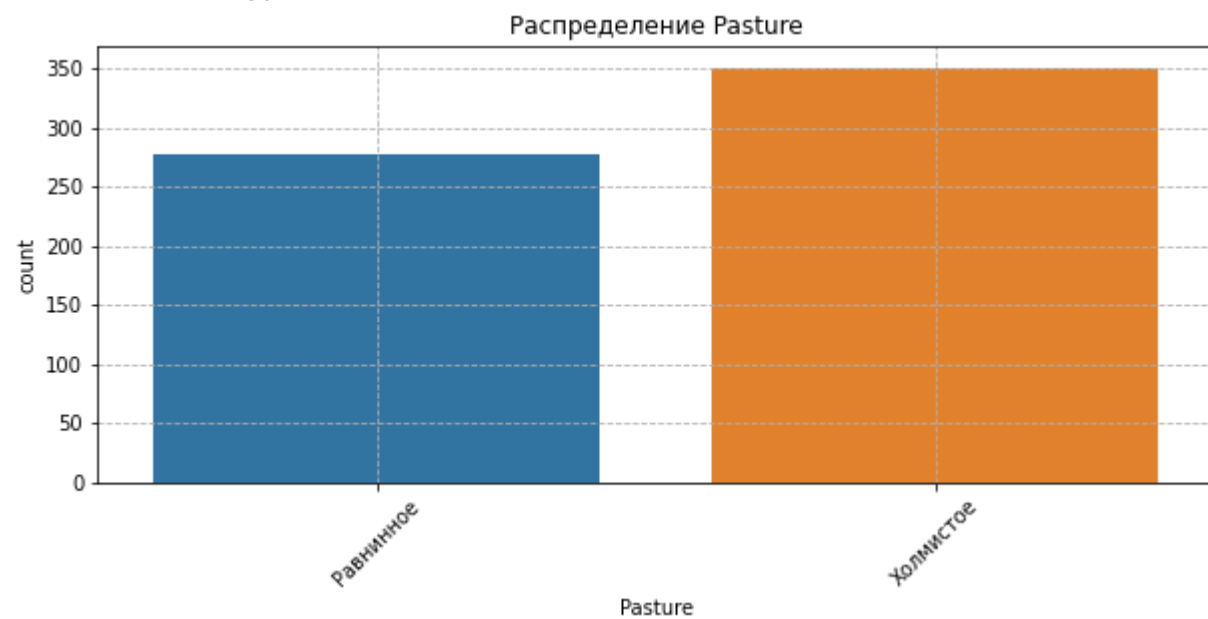
Flavor: 0 выбросов



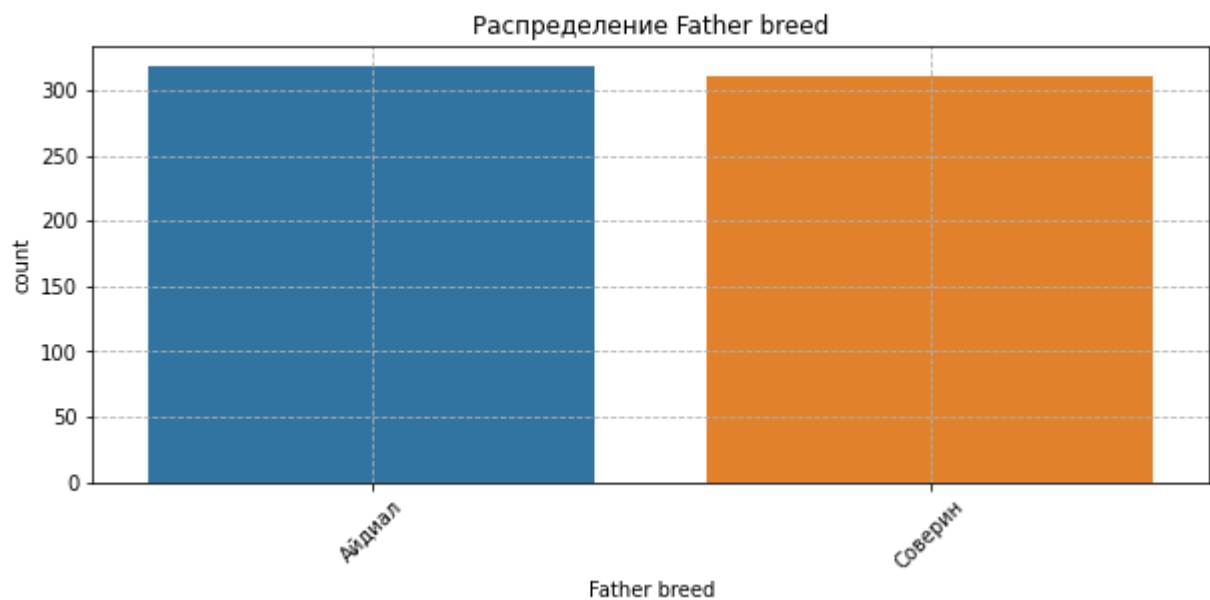
Age: 110 выбросов



Вис Бик Айдиал 373
 РефлешСоверинг 256
 Name: Breed, dtype: int64



Холмистое 351
 Равнинное 278
 Name: Pasture, dtype: int64



```
Айдиал      318
Соверин     311
Name: Father breed, dtype: int64
```

Распределение Milk - нормальное, выбросов нет, средние значения от 6000 до 6500.

Распределение Energy - нормальное, выбросов нет, преобладают более высокие значения.

Распределение Protein - нормальное, выбросов нет, преобладают низкие значения.

Распределение SPO - разделяются на 3 кластера, 0.84 - 0.86, 0.87 - 0.91, 0.93 - 0.96, выбросов нет.

Распределение Fat - неравномерное, выбросов нет, большинство коров имеют высокое содержание жирности в молоке.

Распределение Prot in milk - равномерное, с пробелами, выбросов нет.

Flavor - значения "1" (вкусно) преобладают.

Age - значения "1" (более 2 лет) преобладают значительно.

Breed - преобладает порода "Вис Бик Айдиал".

Pasture - преобладает "Холмистое".

Father breed - преобладает Айдиал, но распределение практически равномерное.

4. Корреляционный анализ

Приступаем к корреляционному анализу. Сделаем матрицу корреляций между всеми признаками.

In [22]:

```
num_cols = ferma_main.select_dtypes(include = 'number').columns.tolist()
cat_cols = ferma_main.select_dtypes(include = 'object').columns.tolist()

ferma_main_no_id = ferma_main.drop(columns = ['id'])
ferma_main_phik = ferma_main_no_id.phik_matrix()

plt.figure(figsize = (15, 7))
sns.heatmap(
    data = ferma_main_phik,
    annot = True,
    fmt = '0.2f'
);
```

```
interval columns not set, guessing: ['Milk', 'Energy', 'Protein', 'SP0', 'Fat', 'Prot i
n milk', 'Flavor', 'Age']
```



Наиболее сильная корреляция у Milk(Удой) - Energy, Protein, SPO, Father Breed, Fat, Age.
 У Flavor(Вкус) - SPO и немного Energy и Pasture, но нужно проверять оба признака дополнительно.

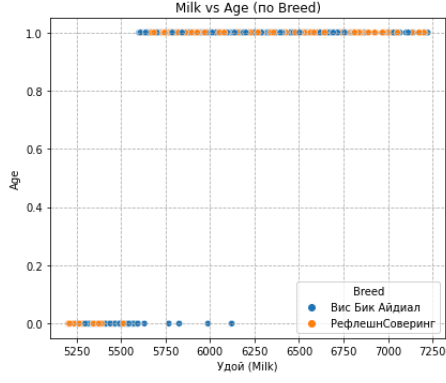
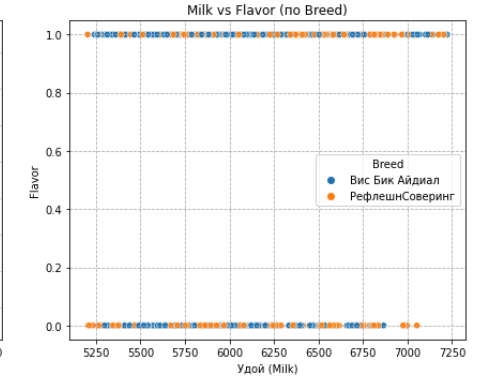
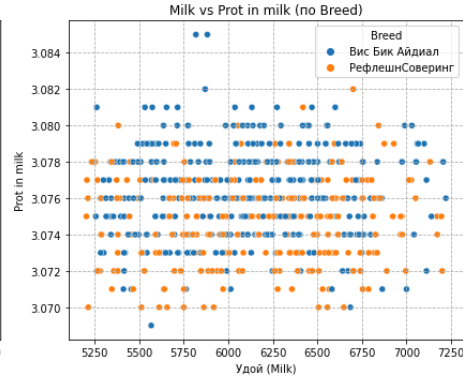
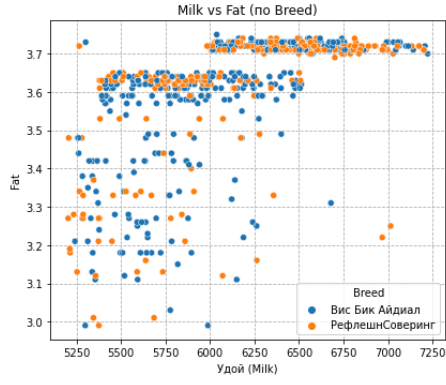
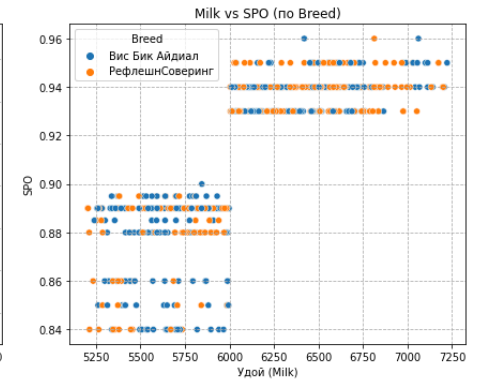
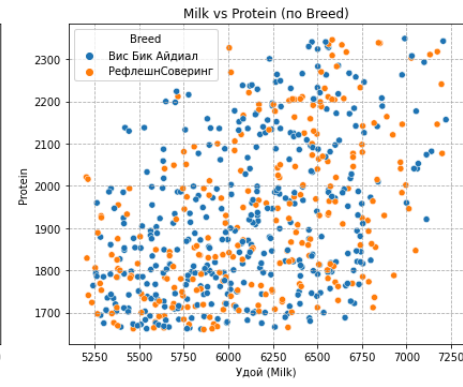
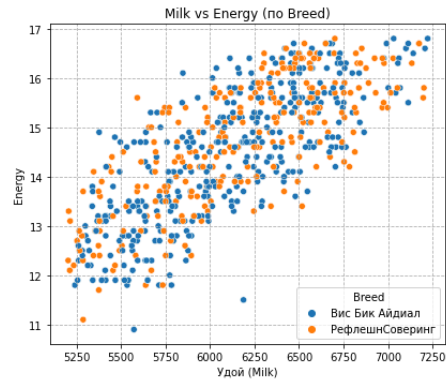
Строим график scatterplot.

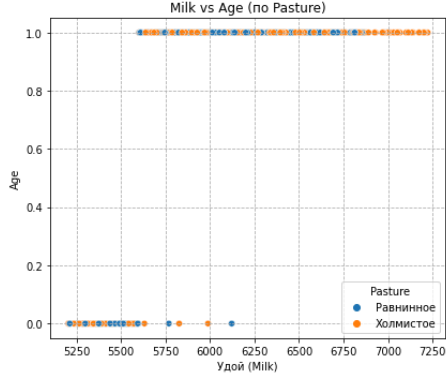
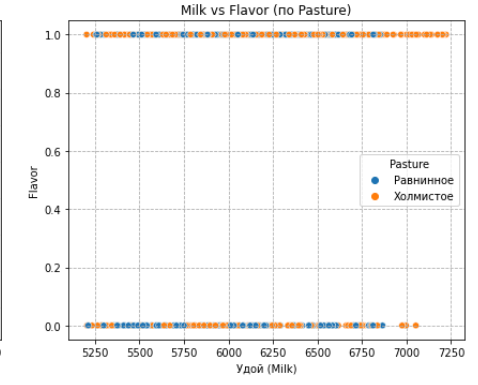
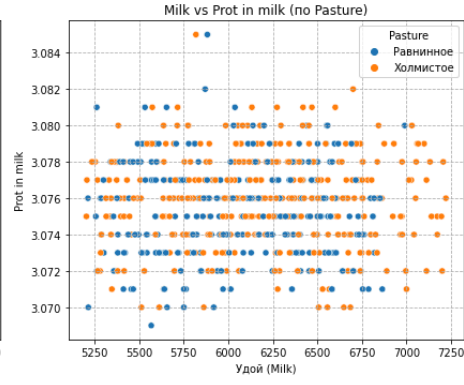
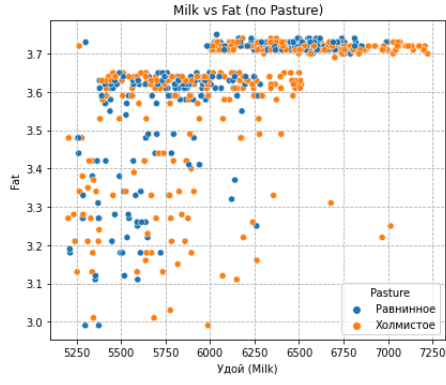
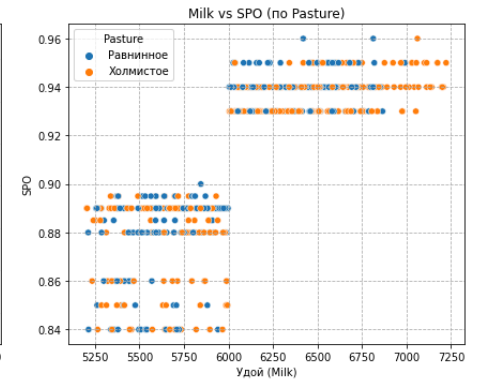
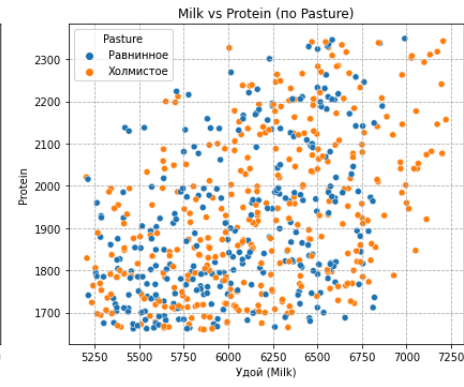
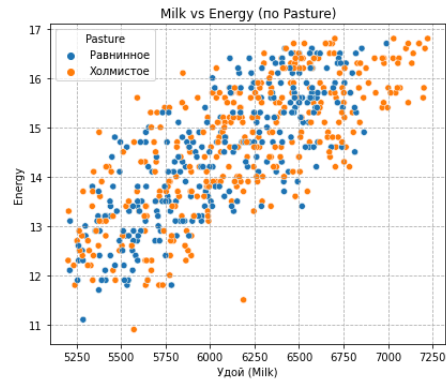
In [23]:

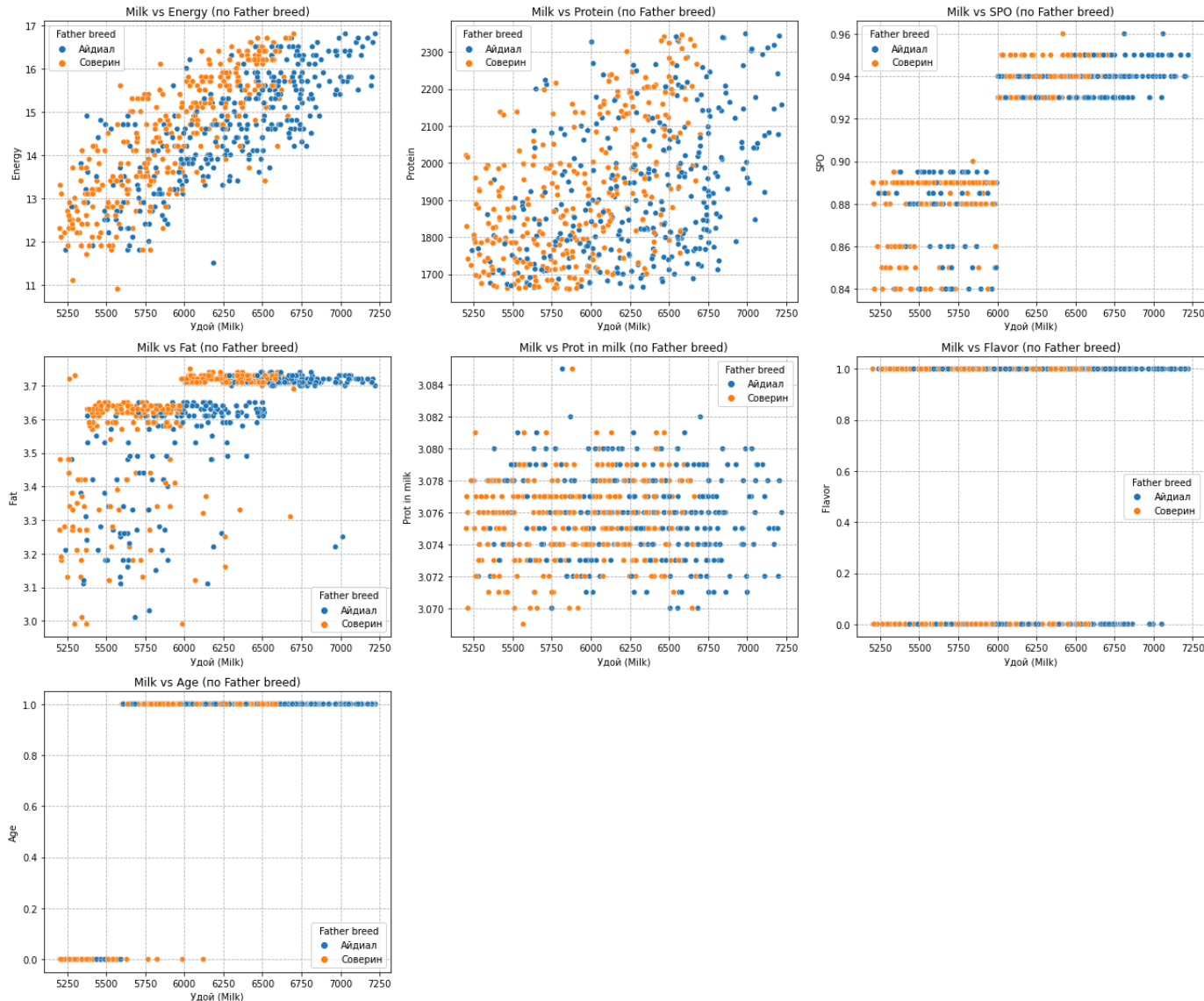
```
num_cols = ferma_main_no_id.select_dtypes(include = 'number').columns.tolist()
cat_cols = ferma_main_no_id.select_dtypes(include = 'object').columns.tolist()
num_cols = [col for col in num_cols if col != 'Milk']

for cat in cat_cols:
    n_plots = len(num_cols)
    n_rows = math.ceil(n_plots / 3)
    plt.figure(figsize=(18, 5 * n_rows))

    for i, num in enumerate(num_cols, 1):
        plt.subplot(n_rows, 3, i)
        sns.scatterplot(
            data = ferma_main_no_id,
            x = 'Milk',
            y = num,
            hue = cat)
        plt.title(f'Milk vs {num} (no {cat})')
        plt.xlabel('Удой (Milk)')
        plt.ylabel(num)
        plt.grid(True, linestyle = '--')
    plt.tight_layout()
    plt.show()
```







Промежуточный вывод:

Milk - Energy:

- линейная зависимость, нормальное распределение, порода папы быка "Айдиал" эффективнее реагирует на повышение Energy(энергетическая кормовая единица). Тип пастбища особо не влияет на удой, но более высокие значения достигаются на холмистом типе.

Milk - Protein:

- нормальное распределение, нет линейной связи, опять же у породы папы быка "Айдиал" более высокие значения удоя, на холмистом типе.

Milk - SPO:

- кластерное распределение, с четкой границей (от 0.84 до 0.9 удой до 6000 кг, от 0.93 до 0.96 удой выше 6 кг), аналогично порода папы быка "Айдиал" и тип пастбища холмистое.

Milk - Fat:

- нелинейное распределение, с отдельными кластерами. От 3.0 до 3.5 наименьшее распределение, 3.6 и отдельно 3.7 наибольшие значения по количеству. Порода папы быка и тип пастбища, аналогично.

Milk - Prot in milk:

- нормальное распределение, связи нет, у породы "Вис Бик Айдиал", в большинстве случаев, более высокое значение. Это же подтверждается по породе папы быка и типу пастбища, аналогично предыдущим наблюдениям.

Milk - Flavor:

- равномерное распределение, связи нет, вкусное молоко вместе БОльшим удоем, встречается чаще по породе папы быка "Айдиал" и на холмистом типе пастбища.

Milk - Age:

- линейная связь, распределение нормальное, больше половины значений имеют значение 1(старше 2 лет), остальное аналогично.

Вывод: Идеальная модель - порода "Вис Бик Айдиал", порода папы быка "Айдиал", тип пастбища "Холмистое". А так же на Удой влияет увеличение Energy(ЭКО) и возраст коровы более 2 лет.

Далее делаем проверку на мультиколлинеарность.

In [24]:

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

numeric_data = ferma_main_no_id.select_dtypes(include=['number']).drop(columns=['Milk'])

vif_data = pd.DataFrame()
vif_data["feature"] = numeric_data.columns
vif_data["VIF"] = [variance_inflation_factor(numeric_data.values, i)
                    for i in range(numeric_data.shape[1])]

print("VIF анализ:")
display(vif_data.sort_values("VIF", ascending=False))
```

VIF анализ:

	feature	VIF
2	SPO	2157.403684
4	Prot in milk	1364.138637
3	Fat	884.359094
0	Energy	333.751615
1	Protein	142.410618
6	Age	9.050413
5	Flavor	2.583714

In [25]:

```
# удаляем лишние признаки
ferma_clean = ferma_main_no_id.drop(columns = ['Fat', 'Prot in milk', 'Protein', 'Flavor'])
```

In [26]:

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

numeric_data = ferma_clean.select_dtypes(include=['number']).drop(columns=['Milk'])

vif_data = pd.DataFrame()
vif_data["feature"] = numeric_data.columns
vif_data["VIF"] = [variance_inflation_factor(numeric_data.values, i)
                    for i in range(numeric_data.shape[1])]

print("VIF анализ:")
display(vif_data.sort_values("VIF", ascending=False))
```

VIF анализ:

	feature	VIF
0	Energy	267.594467
1	SPO	238.859994
2	Age	7.567846

Мультиколлинеарность устранена.

5. Обучение модели линейной регрессии

In [27]:

```
model_one = LinearRegression()

X_one = ferma_clean.drop('Milk', axis = 1)
y_one = ferma_clean['Milk']

X_train_one, X_test_one, y_train_one, y_test_one = train_test_split(
    X_one, y_one,
    test_size = 0.25,
    random_state = 42
)

num_cols_one = [col for col in X_one.select_dtypes('number').columns if col != 'Milk']
cat_cols_one = X_one.select_dtypes('object').columns.tolist()

encoder_one = OneHotEncoder()
encoder_one.fit(X_train_one[cat_cols_one])
X_train_one_ohe = encoder_one.transform(X_train_one[cat_cols_one])
X_test_one_ohe = encoder_one.transform(X_test_one[cat_cols_one])

scaler_one = StandardScaler()
scaler_one.fit(X_train_one[num_cols_one])
X_train_one_sca = scaler_one.transform(X_train_one[num_cols_one])
X_test_one_sca = scaler_one.transform(X_test_one[num_cols_one])

X_train_one = hstack([X_train_one_ohe, X_train_one_sca])
X_test_one = hstack([X_test_one_ohe, X_test_one_sca])

model_one.fit(X_train_one, y_train_one)

predictions_one = model_one.predict(X_test_one)
print('R2 первой модели:', r2_score(y_test_one, predictions_one).round(3))
```

R2 первой модели: 0.808

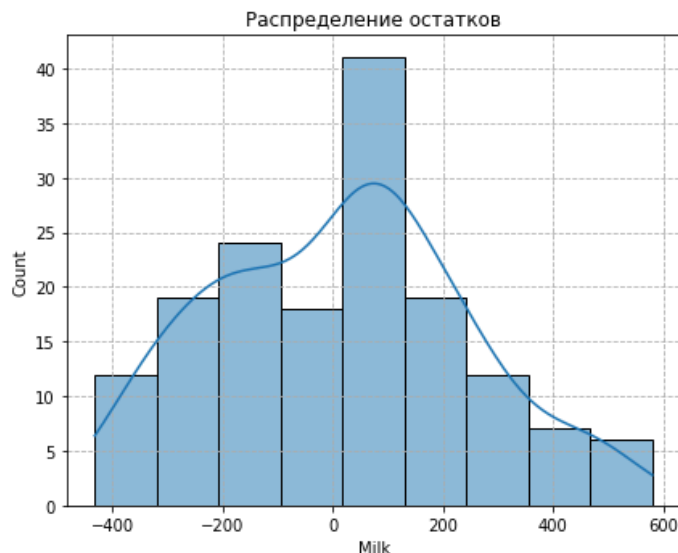
Посмотрим остатки первой модели.

In [28]:

```
residuals_one = y_test_one - predictions_one

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.histplot(residuals_one, kde=True)
plt.title('Распределение остатков')
plt.grid(True, linestyle = '--')

plt.subplot(1, 2, 2)
plt.scatter(predictions_one, residuals_one)
plt.axhline(0, color='red')
plt.title('Остатки vs Предсказания')
plt.tight_layout()
plt.grid(True, linestyle = '--')
```



Распределение относительно равномерное.

Строим вторую модель.

```
In [29]: # сделаем бинарное значение в SP0 и возведем в квадрат Energy
ferma_clean_two = ferma_clean
ferma_clean_two['SP0_2'] = (ferma_clean_two['SP0'] > 0.9).astype(int)
ferma_clean_two = ferma_clean_two.drop(columns = 'SP0')
ferma_clean_two['Energy'] = ferma_clean_two['Energy'] **2
```

```
In [30]: model_two = LinearRegression()

X_two = ferma_clean_two.drop('Milk', axis = 1)
y_two = ferma_clean_two['Milk']

X_train_two, X_test_two, y_train_two, y_test_two = train_test_split(
    X_two, y_two,
    test_size = 0.25,
    random_state = 42
)

num_cols_two = [col for col in X_two.select_dtypes('number').columns if col != 'Milk']
cat_cols_two = X_two.select_dtypes('object').columns.tolist()

encoder_two = OneHotEncoder()
encoder_two.fit(X_train_two[cat_cols_two])
X_train_two_ohe = encoder_two.transform(X_train_two[cat_cols_two])
X_test_two_ohe = encoder_two.transform(X_test_two[cat_cols_two])

scaler_two = StandardScaler()
scaler_two.fit(X_train_two[num_cols_two])
X_train_two_sca = scaler_two.transform(X_train_two[num_cols_two])
X_test_two_sca = scaler_two.transform(X_test_two[num_cols_two])

X_train_two = hstack([X_train_two_ohe, X_train_two_sca])
X_test_two = hstack([X_test_two_ohe, X_test_two_sca])

model_two.fit(X_train_two, y_train_two)

predictions_two = model_two.predict(X_test_two)
print('R2 второй модели:', r2_score(y_test_two, predictions_two).round(3))
```

R2 второй модели: 0.835

```
In [31]: residuals_two = y_test_two - predictions_two

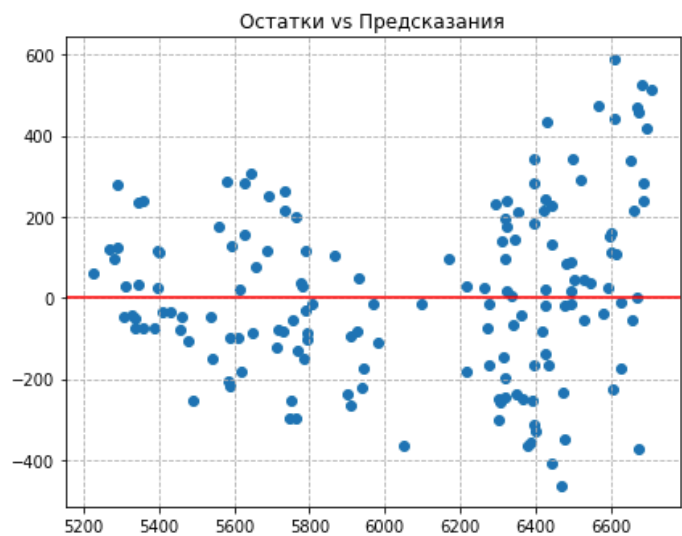
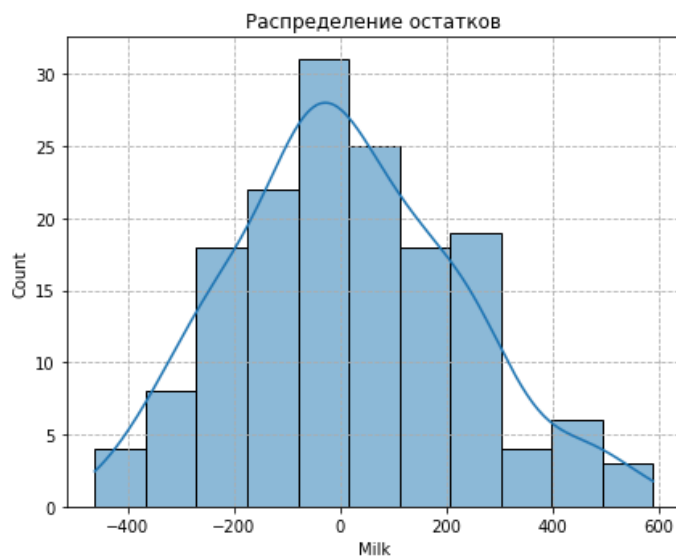
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
```

```

sns.histplot(residuals_two, kde=True)
plt.title('Распределение остатков')
plt.grid(True, linestyle = '--')

plt.subplot(1, 2, 2)
plt.scatter(predictions_two, residuals_two)
plt.axhline(0, color='red')
plt.title('Остатки vs Предсказания')
plt.tight_layout()
plt.grid(True, linestyle = '--')

```



R2 вырос. Распределение аналогичное.

Создаем третью модель.

In [32]:

```

# добавим таблицу ferma_dad
ferma_clean_three = ferma_clean_two
ferma_clean_three['id'] = ferma_main['id']
ferma_clean_three = ferma_clean_three.merge(ferma_dad, on = 'id', how = 'outer')
ferma_clean_three = ferma_clean_three.drop(columns = ['id'])

```

In [33]:

```

model_three = LinearRegression()

X_three = ferma_clean_three.drop('Milk', axis = 1)
y_three = ferma_clean_three['Milk']

X_train_three, X_test_three, y_train_three, y_test_three = train_test_split(
    X_three, y_three,
    test_size = 0.25,
    random_state = 42
)

num_cols_three = [col for col in X_three.select_dtypes('number').columns if col != 'Milk']
cat_cols_three = X_three.select_dtypes('object').columns.tolist()

encoder_three = OneHotEncoder()
encoder_three.fit(X_train_three[cat_cols_three])
X_train_three_ohe = encoder_three.transform(X_train_three[cat_cols_three])
X_test_three_ohe = encoder_three.transform(X_test_three[cat_cols_three])

scaler_three = StandardScaler()
scaler_three.fit(X_train_three[num_cols_three])
X_train_three_sca = scaler_three.transform(X_train_three[num_cols_three])
X_test_three_sca = scaler_three.transform(X_test_three[num_cols_three])

X_train_three = hstack([X_train_three_ohe, X_train_three_sca])
X_test_three = hstack([X_test_three_ohe, X_test_three_sca])

model_three.fit(X_train_three, y_train_three)

```

```
predictions_three = model_three.predict(X_test_three)
print('R2 третьей модели:', r2_score(y_test_three, predictions_three).round(3))
```

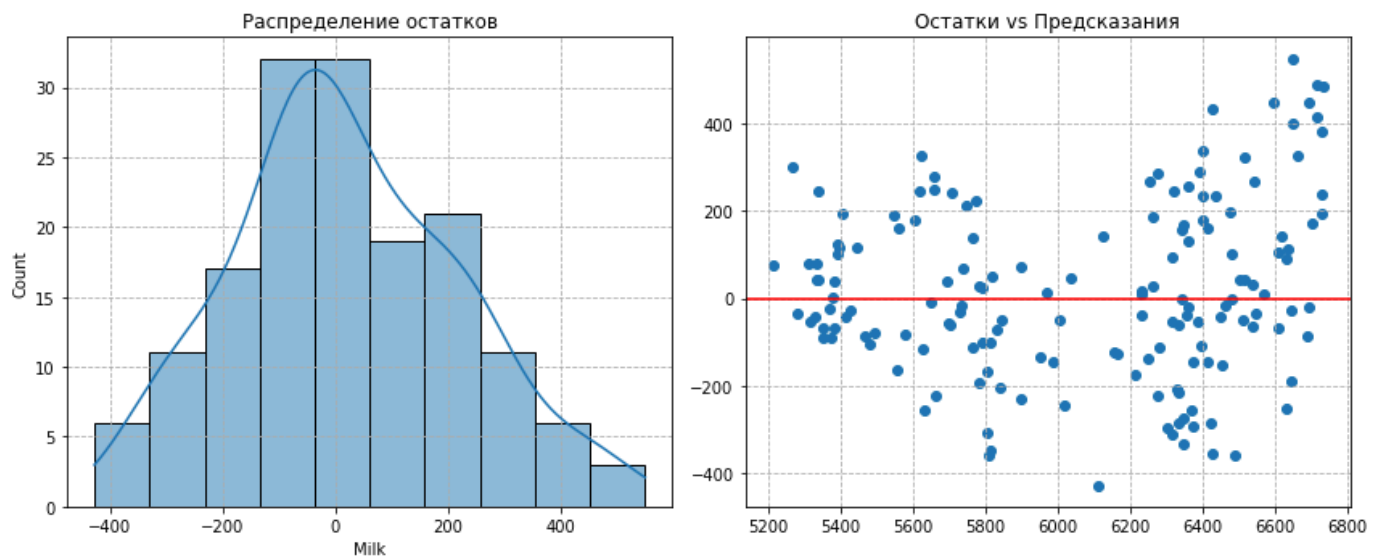
R2 третьей модели: 0.851

In [34]:

```
residuals_three = y_test_three - predictions_three

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.histplot(residuals_three, kde=True)
plt.title('Распределение остатков')
plt.grid(True, linestyle = '--')

plt.subplot(1, 2, 2)
plt.scatter(predictions_three, residuals_three)
plt.axhline(0, color='red')
plt.title('Остатки vs Предсказания')
plt.tight_layout()
plt.grid(True, linestyle = '--')
```



R2 вырос. Остатки аналогичны.

In [35]:

```
def calculate_metrics(y_true, y_pred, model_name):
    return {
        'Модель': model_name,
        'R²': r2_score(y_true, y_pred),
        'MAE': mean_absolute_error(y_true, y_pred),
        'RMSE': mean_squared_error(y_true, y_pred, squared=False)
    }

metrics = [
    calculate_metrics(y_test_one, predictions_one, "Модель 1 (базовая)"),
    calculate_metrics(y_test_two, predictions_two, "Модель 2 (SP0_2 + Energy²)"),
    calculate_metrics(y_test_three, predictions_three, "Модель 3 (с данными о папе)")
]

results = pd.DataFrame(metrics).set_index('Модель')

def highlight_max(s):
    is_max = s == s.max()
    return ['background-color: #98FB98' if v else '' for v in is_max]

styled_results = results.style \
    .format({'R²': '{:.3f}', 'MAE': '{:.1f}', 'RMSE': '{:.1f}'}) \
    .apply(highlight_max, subset=['R²']) \
    .set_caption('Сравнение моделей линейной регрессии') \
    .set_properties(**{'text-align': 'center'})
```

```
display(styled_results)
```

Сравнение моделей линейной регрессии

	R ²	MAE	RMSE
Модель			
Модель 1 (базовая)	0.808	189.3	229.6
Модель 2 (SPO_2 + Energy ²)	0.835	169.2	212.7
Модель 3 (с данными о папе)	0.851	160.6	202.1

Лучшая модель 3.

In [36]:

```
std_error = residuals_three.std()
confidence = 1.96 * std_error

print(f"Средний доверительный интервал: ±{confidence:.1f} кг")
```

Средний доверительный интервал: ±395.6 кг

Далее с помощью 3 модели, с прогнозируем удой коров, которых фермер хочет купить.

In [37]:

```
cow_buy_model_one = cow_buy.drop(columns = ['Fat', 'Prot in milk'])
cow_buy_model_one['Energy'] = ferma_clean_three['Energy'].mean() * 1.05
cow_buy_model_one['SPO_2'] = ((ferma_main['SPO'].mean() * 1.05) > 0.9).astype(int)
```

In [38]:

```
num_cols = ['Energy', 'SPO_2', 'Age']
cat_cols = ['Breed', 'Pasture', 'Father breed', 'Father name']

cow_buy_cat = encoder_three.transform(cow_buy_model_one[cat_cols])
cow_buy_num = scaler_three.transform(cow_buy_model_one[num_cols])
cow_buy_good = hstack([cow_buy_cat, cow_buy_num])

cow_buy_model_one['Milk'] = model_three.predict(cow_buy_good)
```

In [39]:

```
cow_buy_model_one
```

Out [39]:

	Breed	Pasture	Father breed	Father name	Age	Energy	SPO_2	Milk
0	Вис Бик Айдиал	Холмистое	Айдиал	Геркулес	1	223.889831	1	6493.069566
1	Вис Бик Айдиал	Равнинное	Соверин	Буйный	0	223.889831	1	6061.352744
2	РефлешнСоверинг	Равнинное	Соверин	Барин	1	223.889831	1	6239.328574
3	РефлешнСоверинг	Холмистое	Айдиал	Буйный	1	223.889831	1	6544.418690
4	РефлешнСоверинг	Равнинное	Айдиал	Буйный	1	223.889831	1	6488.588746
5	РефлешнСоверинг	Равнинное	Соверин	Геркулес	0	223.889831	1	6094.782641
6	Вис Бик Айдиал	Равнинное	Соверин	Буйный	1	223.889831	1	6459.225702
7	РефлешнСоверинг	Холмистое	Айдиал	Буйный	0	223.889831	1	6146.545732
8	Вис Бик Айдиал	Холмистое	Соверин	Соловчик	1	223.889831	1	6318.548270
9	РефлешнСоверинг	Равнинное	Соверин	Геркулес	1	223.889831	1	6492.655599
10	Вис Бик Айдиал	Холмистое	Айдиал	Геркулес	1	223.889831	1	6493.069566
11	Вис Бик Айдиал	Холмистое	Соверин	Барин	0	223.889831	1	5854.896049
12	РефлешнСоверинг	Холмистое	Айдиал	Буйный	1	223.889831	1	6544.418690
13	Вис Бик Айдиал	Холмистое	Соверин	Соловчик	1	223.889831	1	6318.548270

	Breed	Pasture	Father breed	Father name	Age	Energy	SPO_2	Milk
14	РефлешнСоверинг	Равнинное	Соверин	Геркулес	0	223.889831	1	6094.782641
15	Вис Бик Айдиал	Равнинное	Соверин	Буйный	1	223.889831	1	6459.225702

Прогноз удою для новых коров находится в диапазоне 5 800–6 500 кг/год, что соответствует данным фермы (5 200–7 200 кг).

Лучшие кандидаты (удой >6 300 кг):

Порода: «Вис Бик Айдиал»

Пастбище: Холмистое

Возраст: старше 2 лет

Отец: «Айдиал» (дает +150–200 кг к удою).

6. Обучение модели логистической регрессии

Приступаем к задаче классификации. Основная задача сделать предсказание, будет ли молоко вкусным.

Подготовим датафрейм и посмотрим еще раз корреляцию связанную со вкусом молока.

```
In [40]: # добавим столбец порода папы быка и удалим столбцы с большой мультиколлинеарностью и с
ferma_clean_class = ferma_main
ferma_clean_class = ferma_clean_class.merge(ferma_dad, on = 'id', how = 'outer')
ferma_clean_class = ferma_clean_class.drop(columns = [
    'id', 'Milk', 'Father breed',
    'Father name', 'Breed', 'Prot in milk',
    'Protein'])
```

```
In [41]: ferma_clean_class.phik_matrix()
```

interval columns not set, guessing: ['Energy', 'SPO', 'Fat', 'Flavor', 'Age']

```
Out[41]:
```

	Energy	SPO	Pasture	Fat	Flavor	Age
Energy	1.000000	0.589307	0.103065	0.717846	0.271287	0.768438
SPO	0.589307	1.000000	0.158532	0.644730	0.569538	0.744653
Pasture	0.103065	0.158532	1.000000	0.000000	0.270583	0.129482
Fat	0.717846	0.644730	0.000000	1.000000	0.198952	0.674410
Flavor	0.271287	0.569538	0.270583	0.198952	1.000000	0.166869
Age	0.768438	0.744653	0.129482	0.674410	0.166869	1.000000

```
In [42]: from statsmodels.stats.outliers_influence import variance_inflation_factor

numeric_data_new = ferma_clean_class.select_dtypes(include=['number'])

vif_data = pd.DataFrame()
vif_data["feature"] = numeric_data_new.columns
vif_data["VIF"] = [variance_inflation_factor(numeric_data_new.values, i)
                    for i in range(numeric_data_new.shape[1])]

print("VIF анализ:")
display(vif_data.sort_values("VIF", ascending=False))
```

VIF анализ:

	feature	VIF
1	SPO	844.306156
2	Fat	776.776594
0	Energy	283.629784
4	Age	7.584015
3	Flavor	2.565779

Приступаем к созданию модели.

```
In [43]: X = ferma_clean_class.drop(columns = ['Flavor'])
y = ferma_clean_class['Flavor']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size = 0.25,
    random_state = 42
)

encoder_class = OneHotEncoder()
scaler_class = StandardScaler()

num_cols_class = ['SPO', 'Fat', 'Energy', 'Age']
cat_cols_class = ['Pasture']

encoder_class.fit(X_train[cat_cols_class])
X_train_ohe = encoder_class.transform(X_train[cat_cols_class])
X_test_ohe = encoder_class.transform(X_test[cat_cols_class])

scaler_class.fit(X_train[num_cols_class])
X_train_sca = scaler_class.transform(X_train[num_cols_class])
X_test_sca = scaler_class.transform(X_test[num_cols_class])

X_train_class = hstack([X_train_ohe, X_train_sca])
X_test_class = hstack([X_test_ohe, X_test_sca])

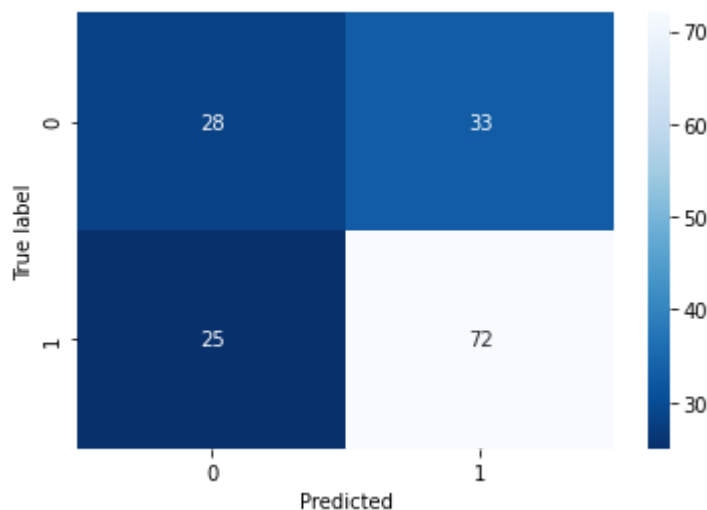
model_class = LogisticRegression()
model_class.fit(X_train_class, y_train)

y_pred = model_class.predict(X_test_class)
y_proba = model_class.predict_proba(X_test_class)
```

```
In [44]: print('Accuracy:', accuracy_score(y_test, y_pred).round(2))
print('Precision:', precision_score(y_test, y_pred).round(2))
print('Recall:', recall_score(y_test, y_pred).round(2))
```

Accuracy: 0.63
Precision: 0.69
Recall: 0.74

```
In [45]: cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues_r')
plt.ylabel('True label')
plt.xlabel('Predicted');
```



По матрице ошибок: ошибка 1 рода, для фермера хуже, так как если молоко окажется невкусным, пострадает репутация.

Accuracy: 0.63 или 63% доля правильных ответов, не плохо, но хотелось бы больше.

Precision: 0.69 или 69% точность предсказаний, что молоко будем вкусным, относительно невкусного.

Recall: 0.74 или 74% точность предсказаний 1 класса, из всех объектов этого класса.

Попробуем свести ошибку 1 рода к нулю, для этого изменим значение порога. Цель увеличить Recall до максимума.

In [46]:

```
y_proba = model_class.predict_proba(X_test_class)[: ,1]

precision, recall, thresholds = precision_recall_curve(y_test, y_proba)
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-9)
optimal_idx = np.argmax(f1_scores)
optimal_threshold = thresholds[optimal_idx]
print(f"Оптимальный порог: {optimal_threshold:.2f}")
```

Оптимальный порог: 0.33

Применяем модель с новым порогом, на датафрейме коров, которых хочет купить фермер.

In [47]:

```
cow_buy_class = cow_buy
cow_buy_class['SP0'] = ferma_main['SP0'].mean() * 1.05
cow_buy_class['Energy'] = ferma_main['Energy'].mean() * 1.05

num_cols_class_cow = ['SP0', 'Fat', 'Energy', 'Age']
cat_cols_class_cow = ['Pasture']

X_cow_buy_cat = encoder_class.transform(cow_buy_class[cat_cols_class])
X_cow_buy_num = scaler_class.transform(cow_buy_class[num_cols_class])
X_cow_buy = hstack([X_cow_buy_cat, X_cow_buy_num])

y_proba_cow_buy = model_class.predict_proba(X_cow_buy)[: , 1]

optimal_threshold = 0.33

cow_buy_class['Flavor_pred'] = (y_proba_cow_buy >= optimal_threshold).astype(int)
```

Добавили в датафрейм столбец с предсказанием вкуса молока. А так же из предыдущего датафрейма добавим расчет удою.

In [49]:

```
cow_buy_final = cow_buy
cow_buy_final['Flavor'] = cow_buy_class['Flavor_pred']
cow_buy_final['Milk'] = cow_buy_model_one['Milk']
cow_buy_final = cow_buy_final.drop(columns = ['Flavor_pred'])
cow_buy_final
```

Out [49]:

	Breed	Pasture	Father breed	Father name	Fat	Prot in milk	Age	SPO	Energy	Flavor
0	Вис Бик Айдиал	Холмистое	Айдиал	Геркулес	3.58	3.076	1	0.958772	15.271073	1 649
1	Вис Бик Айдиал	Равнинное	Соверин	Буйный	3.54	3.081	0	0.958772	15.271073	1 606
2	РефлешнСоверинг	Равнинное	Соверин	Барин	3.59	3.074	1	0.958772	15.271073	1 623
3	РефлешнСоверинг	Холмистое	Айдиал	Буйный	3.40	3.061	1	0.958772	15.271073	1 654
4	РефлешнСоверинг	Равнинное	Айдиал	Буйный	3.64	3.074	1	0.958772	15.271073	1 648
5	РефлешнСоверинг	Равнинное	Соверин	Геркулес	3.63	3.053	0	0.958772	15.271073	1 609
6	Вис Бик Айдиал	Равнинное	Соверин	Буйный	3.57	3.079	1	0.958772	15.271073	1 648
7	РефлешнСоверинг	Холмистое	Айдиал	Буйный	3.40	3.079	0	0.958772	15.271073	1 614
8	Вис Бик Айдиал	Холмистое	Соверин	Соловчик	3.73	3.055	1	0.958772	15.271073	1 637
9	РефлешнСоверинг	Равнинное	Соверин	Геркулес	3.73	3.073	1	0.958772	15.271073	1 649
10	Вис Бик Айдиал	Холмистое	Айдиал	Геркулес	3.68	3.046	1	0.958772	15.271073	1 649
11	Вис Бик Айдиал	Холмистое	Соверин	Барин	3.42	3.075	0	0.958772	15.271073	1 585
12	РефлешнСоверинг	Холмистое	Айдиал	Буйный	3.40	3.065	1	0.958772	15.271073	1 654
13	Вис Бик Айдиал	Холмистое	Соверин	Соловчик	3.73	3.079	1	0.958772	15.271073	1 637
14	РефлешнСоверинг	Равнинное	Соверин	Геркулес	3.77	3.053	0	0.958772	15.271073	1 609
15	Вис Бик Айдиал	Равнинное	Соверин	Буйный	3.34	3.069	1	0.958772	15.271073	1 648

7. Итоговые выводы

In [53]:

```
best_cows = cow_buy[(cow_buy_final['Milk'] > 6300) & (cow_buy_final['Flavor'] == 1)]
display('Рекомендация купить:', best_cows)
```

'Рекомендация купить: '

	Breed	Pasture	Father breed	Father name	Fat	Prot in milk	Age	SPO	Energy	Flavor_pred
0	Вис Бик Айдиал	Холмистое	Айдиал	Геркулес	3.58	3.076	1	0.958772	15.271073	1
3	РефлешнСоверинг	Холмистое	Айдиал	Буйный	3.40	3.061	1	0.958772	15.271073	1
4	РефлешнСоверинг	Равнинное	Айдиал	Буйный	3.64	3.074	1	0.958772	15.271073	1
6	Вис Бик Айдиал	Равнинное	Соверин	Буйный	3.57	3.079	1	0.958772	15.271073	1
8	Вис Бик Айдиал	Холмистое	Соверин	Соловчик	3.73	3.055	1	0.958772	15.271073	1
9	РефлешнСоверинг	Равнинное	Соверин	Геркулес	3.73	3.073	1	0.958772	15.271073	1
10	Вис Бик Айдиал	Холмистое	Айдиал	Геркулес	3.68	3.046	1	0.958772	15.271073	1
12	РефлешнСоверинг	Холмистое	Айдиал	Буйный	3.40	3.065	1	0.958772	15.271073	1
13	Вис Бик Айдиал	Холмистое	Соверин	Соловчик	3.73	3.079	1	0.958772	15.271073	1
15	Вис Бик Айдиал	Равнинное	Соверин	Буйный	3.34	3.069	1	0.958772	15.271073	1

Рекомендации по покупке коров:

1. Подходящие коровы: 10 из 16.
2. Лучшие параметры:
 - Порода: Вис Бик Айдиал.

- Пастбище: Холмистое.
- Отец: Айдиал.
- Возраст: >2 лет.

3. **Ожидаемый удой:** 6000–6500 кг/год.

4. **Риски:**

- 1–2 коровы могут дать молоко «неидеального» вкуса (но это редко).

Действия:

- Купить отобранных 10 коров.
- Увеличить ЭКЕ корма на 5%.
- Контролировать СПО (0.9).